5

Exploring WinUI Controls

WinUI 3 and the Windows App SDK offer many controls and APIs for developers building desktop applications for Windows. The WinUI controls include some controls not previously available to Windows developers, as well as updated controls that were already available in WinUI 2.x and **UWP**. Using these new and updated controls with WinUI 3 enables their use in older versions of Windows that did not previously support this full suite of components. Developers can also leverage Windows App SDK APIs to get direct access to Windows features.

In this chapter, we will cover the following topics:

- Learning more about the controls that are available in WinUI 3
- Exploring the **WinUI 3 Gallery** app for Windows to learn about the WinUI controls and design guidance
- How other Windows App SDK APIs provide Windows developers access to features such as power management and notifications
- How to implement `SplitButton` and `TeachingTip` controls in your own apps

By the end of this chapter, you will have a greater understanding of the controls and libraries in WinUI 3. You will also feel comfortable using the WinUI 3 Gallery app for Windows to explore the controls and find samples that demonstrate how to use them.

Technical requirements

To follow along with the examples in this chapter, please reference the *Technical requirements* section of ***Chapter 2***, *Configuring the Development Environment and Creating the Project*.

The source code for this chapter is available on GitHub here: **https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter05**.

Understanding what WinUI offers developers

In ***Chapter 1***, *Introduction to WinUI*, you learned some background information about the origins of WinUI and UWP. That chapter also covered some of the controls available in the various releases of WinUI. Now, it's time to explore a few of these in more detail. Let's start by looking at a list of controls available to developers in WinUI 3:

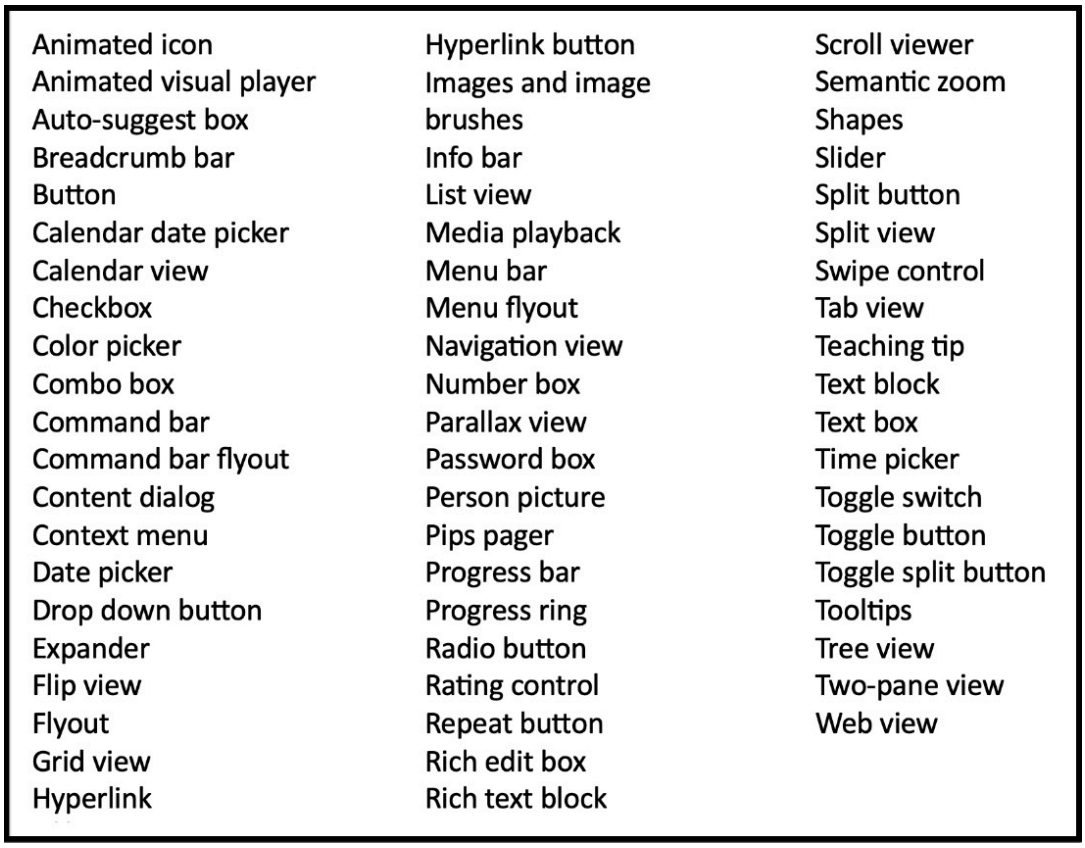

Figure 5.1 – List of WinUI 3 controls

This is quite an extensive list of controls available to developers in the Windows App SDK.

TIP

For an up-to-date list of the available controls, you can check this page on Microsoft Learn: **https://learn.microsoft.com/windows/apps/design/controls/#alphabetical-index**.

If you have developed Windows applications before, most of these control names probably look familiar to you. In the sections ahead, we'll get an overview of some controls that you may not have seen before.

Animated visual player (Lottie)

The **`AnimatedVisualPlayer`** control, shown in *Figure 5.2*, is a WinUI control that can display **Lottie animations**. Lottie is an open source library that can parse and display animations on Windows, the web, iOS, and Android. These animations are created by designers in tools such as **Adobe After Effects** and exported in JSON format. You can learn more about Lottie animations at the official website, **http://airbnb.io/lottie/#/**:

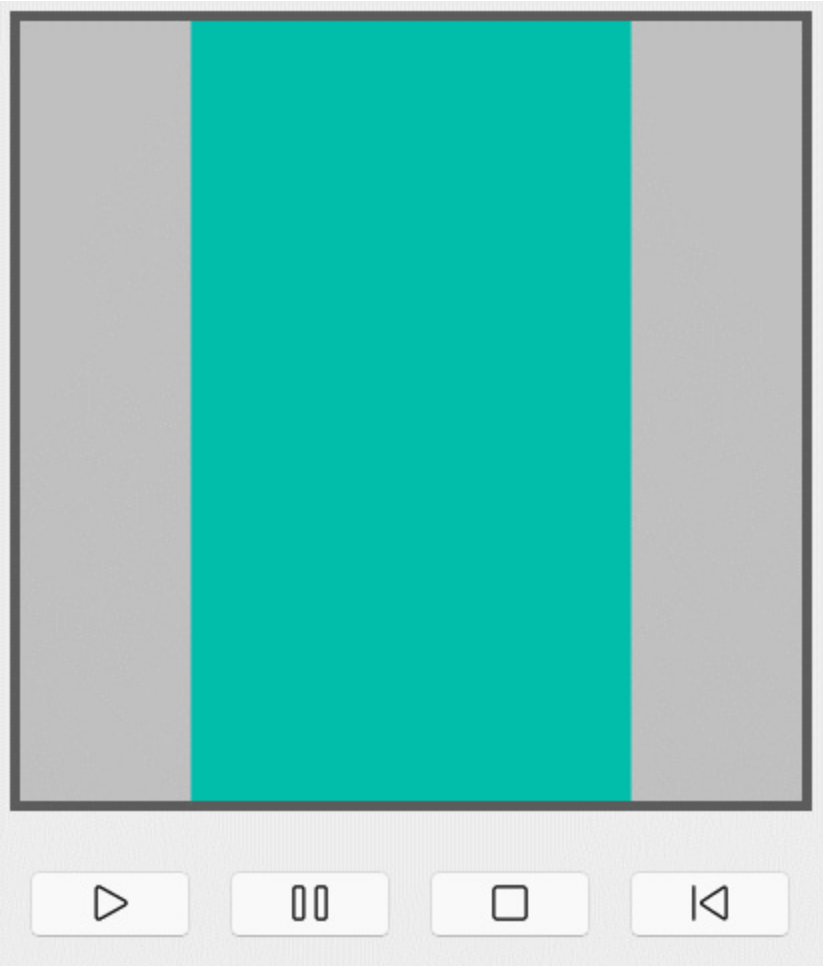

Figure 5.2 – The AnimatedVisualPlayer control

Navigation View

`NavigationView` provides a user-friendly page navigation system. Use it to give users quick access to all your application's top-level pages. **`NavigationView`** can be configured to appear as a menu at the top of the application, with each page's link appearing like a tab across the top of the page:

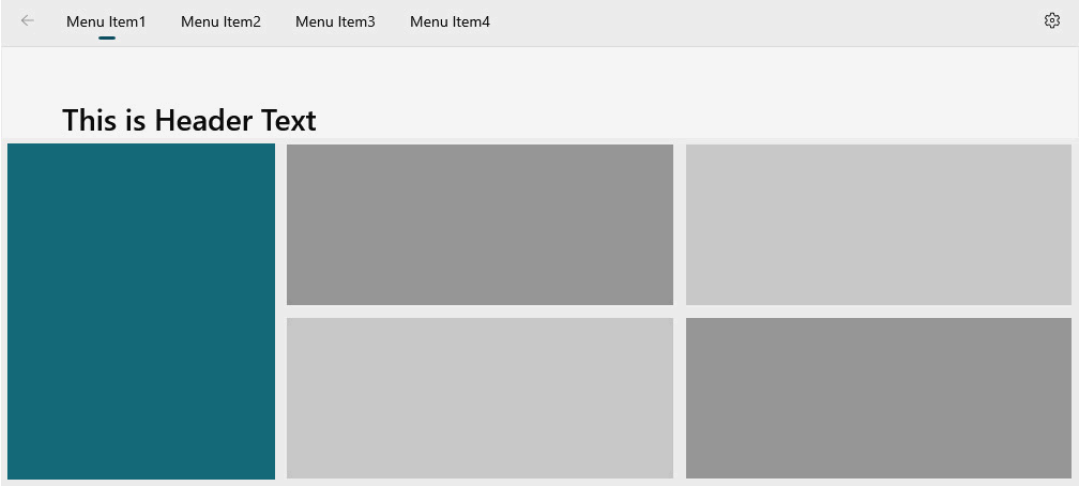

Figure 5.3 – NavigationView configured at the top of a page

`NavigationView` can also be configured to appear on the left-hand side of the page. This is a view that should be familiar to Windows and Android users. This menu format is commonly known as a **hamburger menu**. This is how the view appears when the menu is in a collapsed state:

Figure 5.4 – A collapsed left NavigationView control

In either configuration, `NavigationView` can hide or show a back arrow to navigate to the previous page and has a **Settings** menu item to show the application's settings page. If your application does not have a settings page, this item should be hidden. When the left menu is expanded by clicking the *hamburger* icon below the back arrow, the menu text is displayed, along with the respective icons:

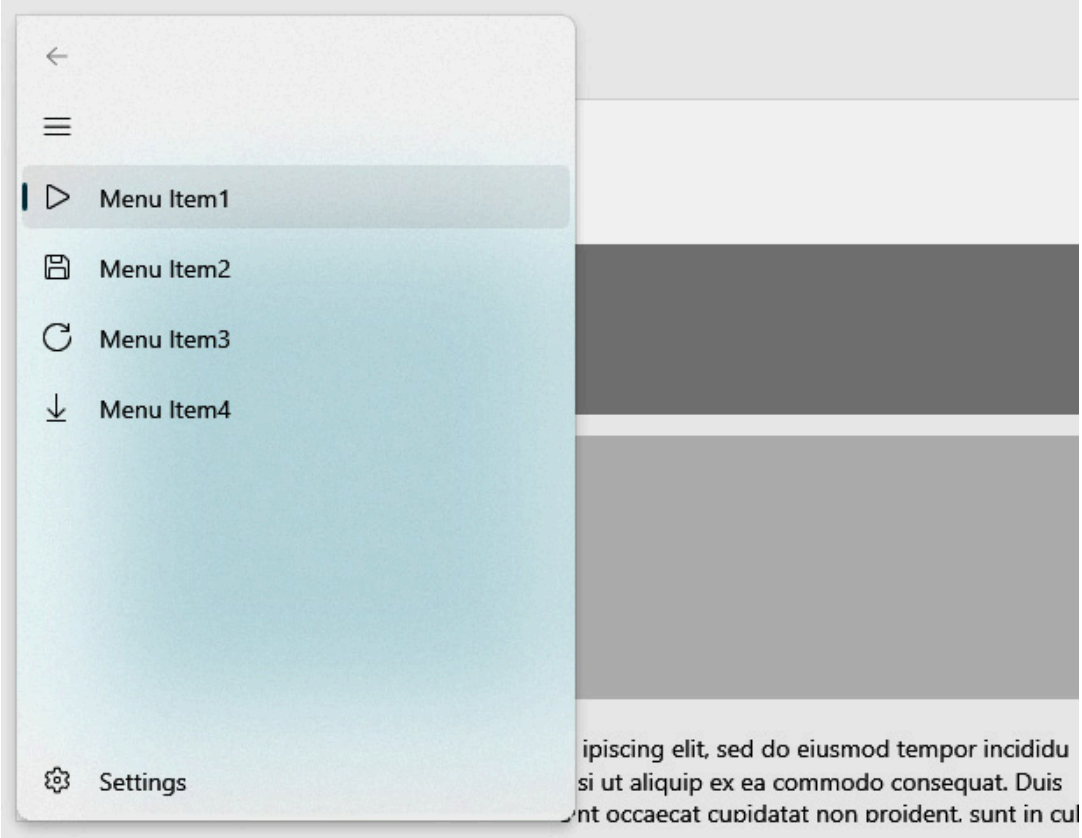

Figure 5.5 – The expanded left NavigationView control

Everything on the menu is configurable. You can group pages with the **Actions** section. You can hide or show the search box, the **More Info** link, and, as we previously mentioned, the **Settings** item. If your application navigates multiple top-level pages, you should absolutely consider using `NavigationView`.

Parallax view

Parallax scrolling is a design concept that links scrolling a list or web page to scrolling a background image or other animations. A great example of a website with parallax scrolling is *History of the Web* (https://webflow.com/ix2). The `ParallaxView` control brings this concept to your WinUI applications. You link `ParallaxView` to `ListView` and a background image, and it will provide a parallax effect when `ListView` is scrolled. There are settings to control the relationship between scrolling the list and the amount the image scrolls. This effect has a great impact on users when it is not overused:

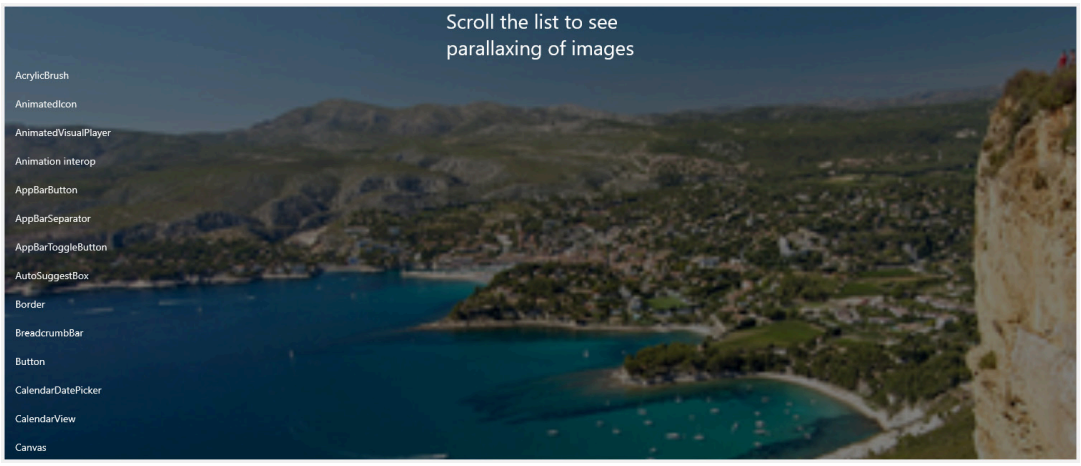

Figure 5.6 – The ParallaxView control scrolled to the top of a list

Figure 5.7 – The ParallaxView control scrolled partially through a list

Rating control

Everyone is familiar with rating controls. You see them on shopping websites, streaming apps, and online surveys. The WinUI **`RatingControl`** control allows users to rate items in your application from 1 to 5 stars:

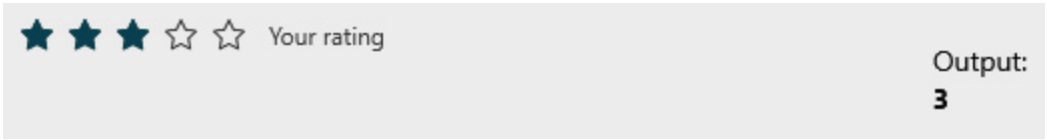

Figure 5.8 – RatingControl displaying the user's rating

The control can also allow you to clear a rating by swiping left on the control and can show a placeholder value before the user has pro-

vided their own rating. Applications frequently use the placeholder value as a means of showing users the average rating given by other users.

Now that we have discussed a few of the controls that were added with WinUI, let's explore a Windows application that makes it easy to explore them on your own.

Exploring the WinUI 3 Gallery app for Windows

Because there are so many powerful and configurable WinUI controls available, the WinUI team at Microsoft decided to create an application that allows Windows developers to explore and even try the controls. **WinUI 3 Gallery** is a great tool to get familiar with the controls, decide which ones are a good fit for your application, and get some sample code and design guidance.

NOTE

There is also a **WinUI 2 Gallery** *app for UWP developers. The two apps used to be a single app called* **XAML Controls Gallery**.

To install the WinUI 3 Gallery app, you can visit its Microsoft Store page on the web (**https://apps.microsoft.com/store/detail/winui-3-gallery/9P3JFPWWDZRC**) or launch the Microsoft Store app for Windows and search for **`WinUI 3 Gallery`**. The Gallery app itself is open source. You can browse the code to learn more about it on GitHub: **https://github.com/microsoft/WinUI-Gallery**.

Once it's been installed, launch the application:

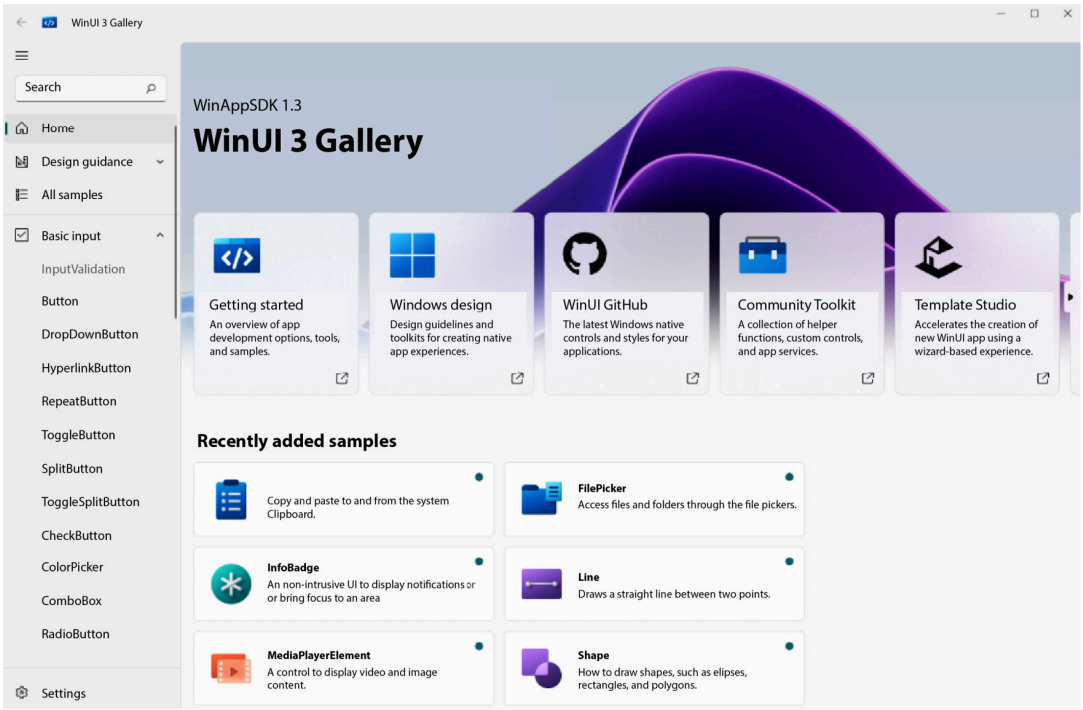

Figure 5.9 – The WinUI 3 Gallery application

In the **Recently added samples** and **Recently updated samples** sections of the application's main page, you can quickly see which controls and samples have been recently added or updated. The application uses a navigation system that should look familiar. The app uses a **`NavigationView`** control, and the left-hand menu allows for quickly browsing or searching the various controls in the gallery.

NOTE

If you search for the controls shown in the previous section, you may notice that the screenshots provided for the controls in this chapter were taken from the WinUI 3 Gallery application.

Learning about the ScrollViewer control

Suppose you were considering adding scrolling capability to a region of a page in your application. In the gallery application, you can click **Scrolling** on the left navigation menu and then click on the **ScrollViewer** card on the **Scrolling** page. That will bring you to the details page for the WinUI **`ScrollViewer`** control:

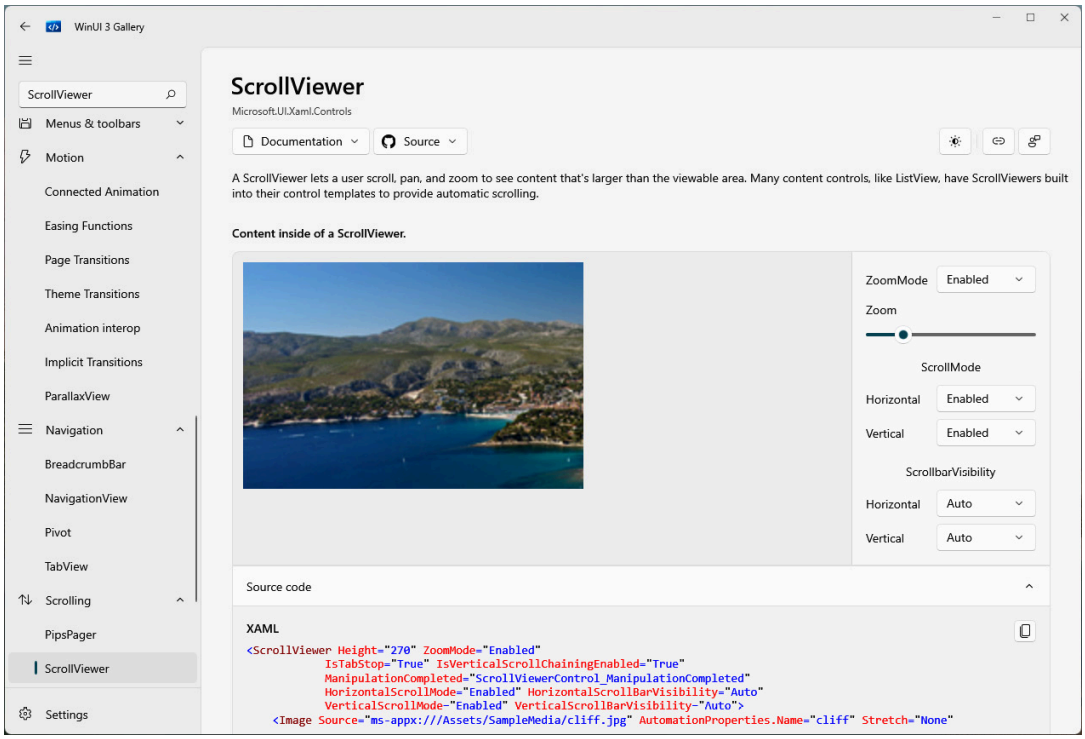

Figure 5.10 – The ScrollViewer control detail page in XAML Controls Gallery

The control details page includes several sections. The header area provides a brief description of the control and its purpose. The right pane provides useful links to online documentation, other related controls in the gallery, and a link to provide feedback on the current gallery page.

The middle area of the page itself contains three sections: a rendered control, a functional control, and a properties panel. Using the properties panel, you can update some of the properties of **`ScrollViewer`** and see the rendered control immediately update. In the bottom center, you will find a panel containing the source code for the rendered control. The source control pane is very handy for copying the code to use as a starting point in your own project.

The gallery application's design also responds well to being resized. If you drag the right-hand side of the window to make it as narrow as possible, you will see the left and right panels collapse, and the center area will realign to a single vertical column:

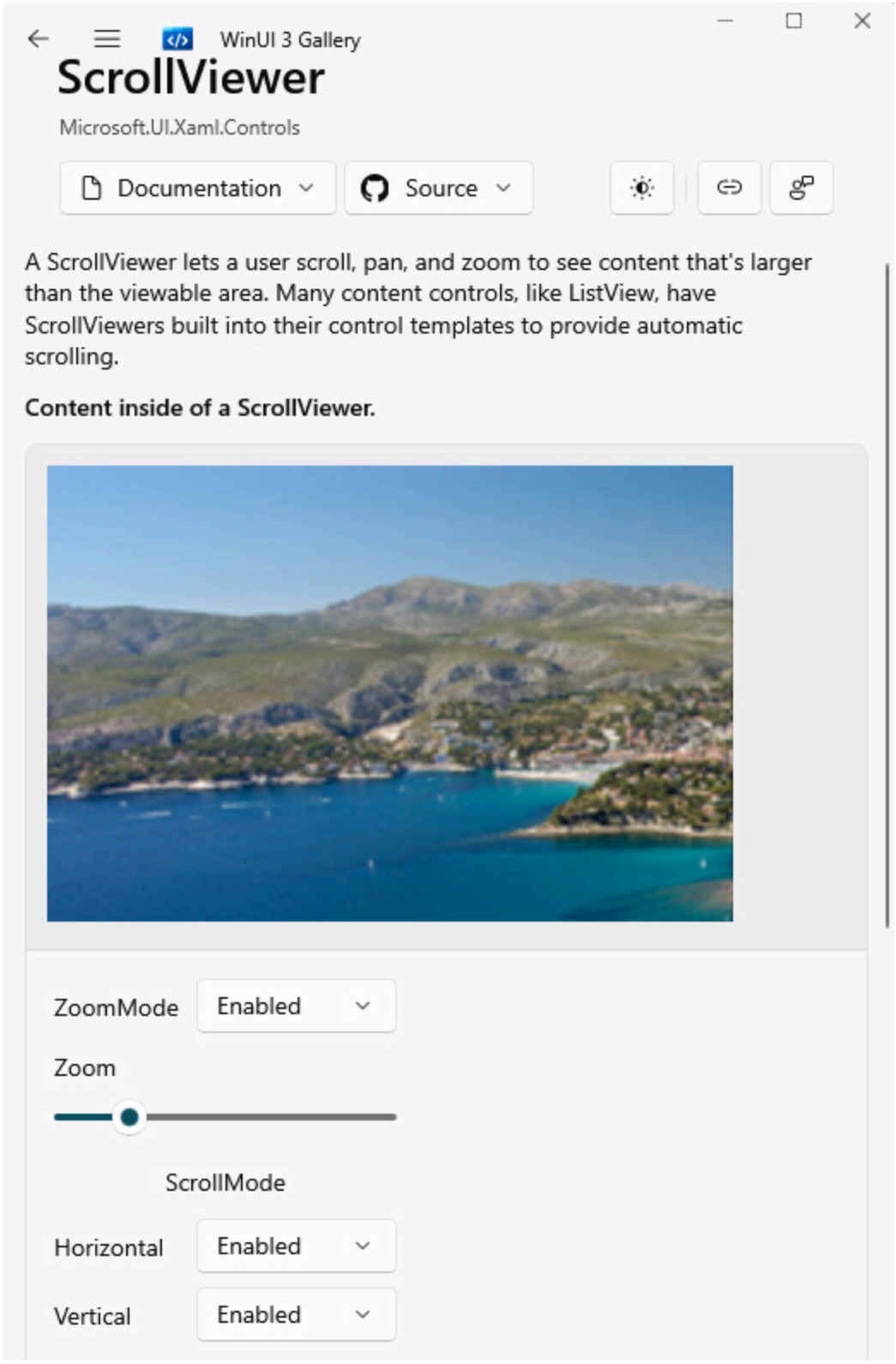

Figure 5.11 – The gallery application resized horizontally

You can imagine this view would fit well on a **Microsoft Surface Go laptop** or other small PC.

Take some time to explore the controls in the gallery. Some of the code samples can be quite lengthy. Try changing some control properties and notice how the XAML code updates to reflect the new property values. This is a great way to learn XAML and familiarize yourself with the WinUI controls.

Now, let's take a tour of the new features of WinUI 3.

Reviewing what's new in WinUI 3 and the Windows App SDK

Although WinUI 3 is a major release, the number of new features, as compared to WinUI 2.x, is not extensive. That may be surprising to many people, but simply creating WinUI 3 and the Windows App SDK as a standalone release was quite an undertaking. We'll look at the most significant features in the following subsections.

Backward compatibility

To make WinUI applications compatible with more versions of Windows (it works with Windows 10, version 1809 and later), the WinUI team had to extract the UWP controls from the Windows SDK and move them to the new `Microsoft.UI.*` libraries in the Windows App SDK. The result of this work not only creates compatibility with more versions of Windows but also enables developers to consume WinUI, regardless of whether they are using .NET or Win32 as the underlying platform. C# developers can build .NET apps with WinUI for Desktop projects, and C++ developers can consume WinUI on the Win32 platform.

Fluent UI and modern look and feel

Developers who maintain **Windows Presentation Form** (**WPF**), WinForms, and **Microsoft Foundation Class** (**MFC**) applications

aren't easily able to achieve the modern look and feel of Windows with **Fluent UI** the way you can in WinUI. We will be covering Fluent UI in depth in ***Chapter 7***, *Fluent Design System for Windows Applications*.

Visual Studio tooling

Visual Studio can now add WinUI project templates without installing a separate extension from the Visual Studio Marketplace. As discussed in the opening chapter, WinUI support can be added along with the **.NET Desktop Development** workload for Visual Studio. Starting a new project with WinUI in Visual Studio is literally as easy as going to **File** | **New Project**.

The WebView2 control

One new control available to developers in WinUI 3 is **`WebView2`**. This new version of **`WebView`** is built on the Chromium-based Microsoft Edge web browser. If you need to embed some web content into your app, **`WebView2`** is the control you should use to ensure maximum compatibility with modern web standards.

Here is a screenshot of **WebView2** running in the **WinUI 3 Gallery** app:

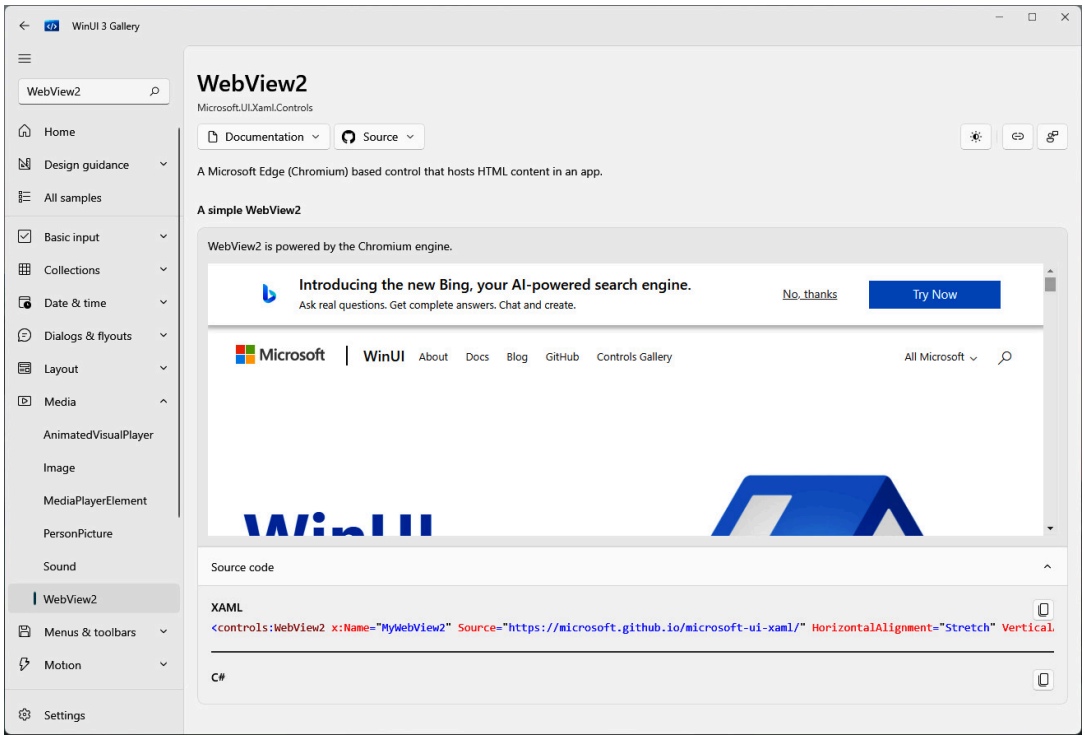

Figure 5.12 – WebView2 running in the WinUI 3 Gallery application

Web content can be loaded into the control from the web or local network, from files in local storage, or from files embedded into the application's binaries. Here are some examples of loading into **`WebView2`** from each type of source:

```
<!-- Load a website. -->
<WebView2 x:Name="WebView_Web"
  Source="https://www.packtpub.com"/>
<!– Load web files from local storage. -->
<WebView2 x:Name="WebView_Local" Source="ms-appdata:
  ///local/site/index.html"/>
<!– Load web files embedded from the app package. -->
<WebView2 x:Name="WebView_Embedded" Source="ms-appx-
  web:///web/index.html"/>
```

If you have an existing application written for the web, then the **`WebView2`** control is a great way to integrate it into your new WinUI client application.

Let's shift gears for a moment and talk about some features of the Windows App SDK outside of the WinUI 3 controls.

What's new in the Windows App SDK

You know by now that WinUI 3 is a part of the Windows App SDK. The Windows App SDK has its own versions, while WinUI 3 will always be known as WinUI 3. The latest stable release of the Windows App SDK at the time of this writing is version 1.4. In this section, we will review some of the Windows App SDK features that are outside the controls in WinUI. For a complete list of the current Windows App SDK features, you can review this page on *Microsoft Learn*: **https://learn.microsoft.com/windows/apps/windows-app-sdk/#windows-app-sdk-features**.

Power management

Your app can subscribe and respond to power management events as part of the Windows App SDK's app lifecycle API. Some of the events exposed by the **`PowerManager`** class in the SDK include the following:

- **`BatteryStatusChanged`**: Raised when the status of the battery in the system has changed. Use the **`BatteryStatus`** property to get the current status.
- **`DisplayStatusChanged`**: Raised when the status of the display where the app is running changes. Use the **`DisplayStatus`** property to get the current status.
- **`EffectivePowerModeChanged`**: Raised when the effective power mode of the system has changed. These can also be referred to as power plans, such as battery saver and high performance. Use the **`EffectivePowerMode`** property to get the current mode.
- **`SystemSuspendStatusChanged`**: Raised when the system is suspended or resumes. In UWP, these types of events were part of the built-in lifecycle in the **`App`** class. Use the **`SystemSuspendStatus`** property to get the current status.

- **`UserPresenceStatusChanged`**: Raised when the system detects a change in the user's presence. Use the **`UserPresenceStatus`** property to get the current status.

A full list of **`PowerManager`** events and properties can be viewed on *Microsoft Learn*: **https://learn.microsoft.com/windows/windows-app-sdk/api/winrt/microsoft.windows.system.power.powermanager**.

Responding to changes in these events can help your app be intelligent in how and when it performs resource-intensive actions. You may have some background processing that should only be performed when a system is plugged in or its battery capacity is above a certain percentage. You could also choose to suspend screen animations or dashboard updates when a user's screen is dimmed or turned off in order to conserve power.

Next, let's discuss the windowing capabilities in the Windows App SDK.

Window management

The Windows App SDK has some limited window management capabilities that can be leveraged by using the **`AppWindow`** class. By using some of the interop APIs, your app can get the **`HWND`** and **`WindowId`** values for the current window. In Win32 development, a **window handle** (or **HWND**) uniquely identifies a window object in the operating system. **`WindowId`** can then be used to get a reference to the current **`AppWindow`** object:

```
var appWindow = Microsoft.UI.Windowing.AppWindow
  .GetFromWindowId(windowId)
```

Most of the properties on **`AppWindow`** are read-only. You can get information such as its size, position, visibility, and the **`WindowId`** value of

its owner. One writable property of `AppWindow` is `Title`. Setting `Title` allows you to change the text in the title bar of the current window.

Push notifications and app notifications

Push notifications can be used to either interact with the app without notifying the user or display a toast notification in Windows. The latter is considered an **app notification** and is most familiar to users. The other raw notifications that trigger within the app are used for things such as waking the app from an inactive state or for data syncing purposes.

The Windows App SDK supports both types of notifications. The APIs for these are in the `Microsoft.Windows.PushNotifications` and `Microsoft.Windows.AppNotifications` namespaces. Exploring notifications is beyond the scope of this chapter, but you can explore some quick starts on *Microsoft Learn* from the *Push notifications overview* page: **https://learn.microsoft.com/windows/apps/windows-app-sdk/notifications/push-notifications/**. We will add notifications to our project in ***Chapter 8***, *Adding Windows Notifications to WinUI Applications*.

Now that we've learned about some of the new features in WinUI and the Windows App SDK, let's get back to our project and add a couple of new controls to it.

Adding some new controls to the project

In this section, we are going to use two controls that are only available to Windows applications with WinUI. We are going to change the **Save** button to `SplitButton` to allow users to save and return to a list of items, or save and continue adding another item to the item details page. Then, we will add a `TeachingTip` control to inform users of the

new saving capabilities. To follow along with these steps, you can use the starter project on GitHub (**https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter05/Start**). Let's start by updating the **Save** button.

Using the SplitButton control

Follow these steps:

1. First, in **`ItemDetailsViewModel`**, add a new **`SaveItemAndContinue`** method to be bound to the **`Click`** event of our new **`SplitButton`** control:

```
public void SaveItemAndContinue()
{
    Save();
    _itemId = 0;
    ItemName = string.Empty;
    SelectedMedium = null;
    SelectedLocation = null;
    SelectedItemType = null;
    IsDirty = false;
}
```

   In the **`SaveItemAndContinue`** method, we are calling **`Save`** and then resetting all the item state data so that it's ready for a new item to be entered. The one problem here is that **`Save`** currently navigates back to the previous page. Let's fix that.
2. To remove the call from **`Save`** to return to the previous page, we need a new method for the **Save** button to use. Create a new method named **`SaveItemAndReturn`**:

```
public void SaveItemAndReturn()
{
    Save();
```

```
    _navigationService.GoBack();
}
```

Here, we are calling **`Save`** and then navigating back to the previous page. The call to **`_navigationService.GoBack`** can now be removed from **`Save`**.

3. We're currently using **`x:Bind`** to bind directly to the save methods instead of using **`ICommand`**. So, you can remove the **`RelayCommand`** attribute from **`Save`** and the **`CanSaveItem`** method from **`ItemDetailsViewModel`**. You will also need to remove the **`NotifyCanExecuteChangedFor`** attribute from the **`isDirty`** private member.
4. Finally, open **`ItemDetailsPage`** and update the **`Save`** button to be a **`SplitButton`** control:

```
<SplitButton x:Name="SaveButton"
             Content="Save and Return"
             Margin="8,8,0,8"
             Click="{x:Bind ViewModel
               .SaveItemAndReturn}"
             IsEnabled="{x:Bind ViewModel.IsDirty,
               Mode=OneWay}">
    <SplitButton.Flyout>
        <Flyout>
          <StackPanel>
            <Button Content="Save and Create New"
                    Click="{x:Bind ViewModel
                      .SaveItemAndContinue}"
                    IsEnabled="{x:Bind
                      ViewModel.IsDirty,
                         Model=OneWay}"
                    Background="Transparent"/>
            <Button Content="Save and Return"
                    Click="{x:Bind ViewModel
                      .SaveItemAndReturn}"
                    IsEnabled="{x:Bind
                      ViewModel.IsDirty, Mode=OneWay}"
                    Background="Transparent"/>
```

```
            </StackPanel>
          </Flyout>
      </SplitButton.Flyout>
  </SplitButton>
```

The content of **`SplitButton`** has been updated to **`Save and Return`**. We've also updated the binding to use the **`Click`** event to invoke the action and the **`IsEnabled`** property with **`IsDirty`**. A new child **`Flyout`** item is also added. **`Flyout`** contains a **`StackPanel`** control with **`Button`** controls for both the **Save and Return** and the new **Save and Create New** action, with the **`Click`** event invoking **`SaveItemAndContinue`**.

5. That's it! Now, run the application and try out this new feature. Here's how the new button looks when you click the drop-down arrow:

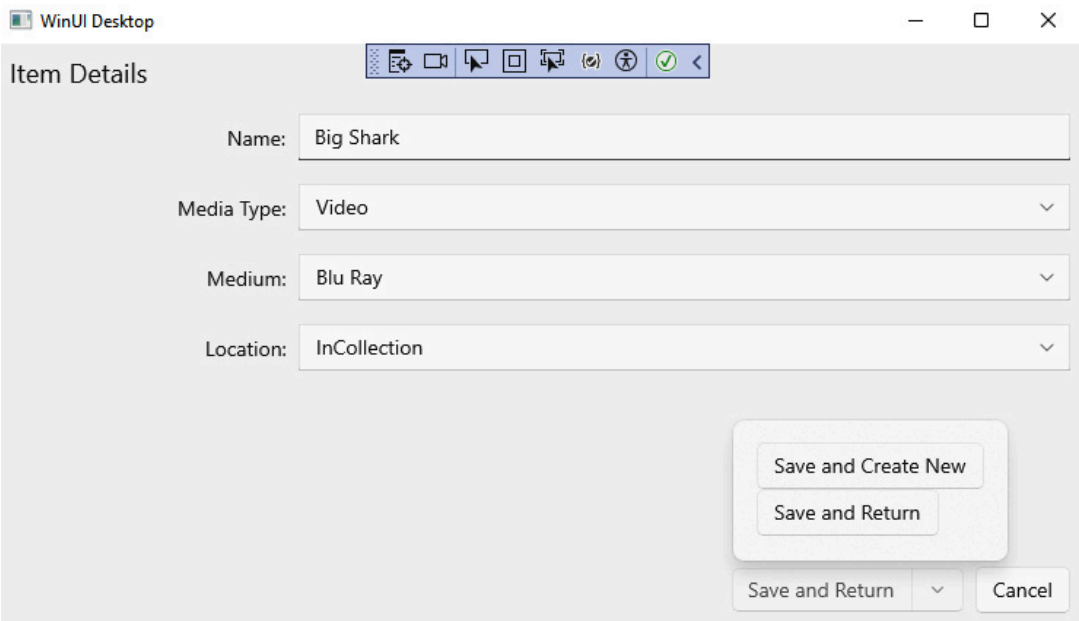

Figure 5.13 – Using the new SplitButton control to save items

Adding a TeachingTip control to the Save button

A **`TeachingTip`** control is a great way to educate users about features in your application. It is a small popup with header text and content

text. You may have seen them in some of the Windows applications you use.

`TeachingTip` can either be linked to a control on the page or it can be placed directly on the page with an optional **`PreferredPlacement`** property, controlling where it appears on the page. It can be configured to be dismissed by the user with a **Close** button or automatically when the user starts interacting with the page.

To add a **`TeachingTip`** control for our **`SplitButton`** control, add it to the **`Resources`** control in **`ItemDetailsPage`**, like so:

```
<SplitButton x:Name="SaveButton" Content="Save and Return"
 Margin="8,8,0,8" Command="{x:Bind ViewModel.SaveCommand}">
    <SplitButton.Flyout>
        <Flyout>
            <Button Content="Save and Create New"
            Command="{x:Bind ViewModel
               .SaveAndContinueCommand}"
                   Background="Transparent"/>
        </Flyout>
    </SplitButton.Flyout>
    <SplitButton.Resources>
        <TeachingTip x:Name="SavingTip"
                        Target="{x:Bind SaveButton}"
                        Title="Save and create new"
                        Subtitle="Use the dropdown button
                        option to save your item and create
                        another.">
        </TeachingTip>
    </SplitButton.Resources>
</SplitButton>
```

Inside the **`TeachingTip`** control, we're binding **`Target`** to **`SaveButton`** and setting **`Title`** and **`Subtitle`** to educate users about the new **Save and create new** feature.

There's an additional call needed in the **`ItemDetailsPage`** constructor to make the tip appear:

```
SavingTip.IsOpen = true;
```

If you run the application now, **`TeachingTip`** is going to appear every time the user opens **`ItemDetailsPage`**. This is going to quickly annoy our users. We can add a little bit of code to **`ItemDetailsPage.xaml.cs`** to save a user-level setting indicating that the current user has already seen this **`TeachingTip`** control. Then, the next time we load the page, we'll check this setting so that the app can skip the code that displays the tip.

We're going to leverage Windows local storage to save and load the user setting:

```
Windows.Storage.ApplicationDataContainer localSettings =
  Windows.Storage.ApplicationData.Current.LocalSettings;
// Load the user setting
string haveExplainedSaveSetting = localSettings.Values
  [nameof(SavingTip)] as string;
// If the user has not seen the save tip, display it
if (!bool.TryParse(haveExplainedSaveSetting, out bool
  result) || !result)
{
    SavingTip.IsOpen = true;
    // Save the teaching tip setting
    localSettings.Values[nameof(SavingTip)] = "true";
}
```

Now, the user will only see this tip the first time they load **`ItemDetailsPage`**.

Let's see our **`TeachingTip`** control in action. Run the application, select an item, and click **Add/Edit Item** to open **`ItemDetailsPage`**:

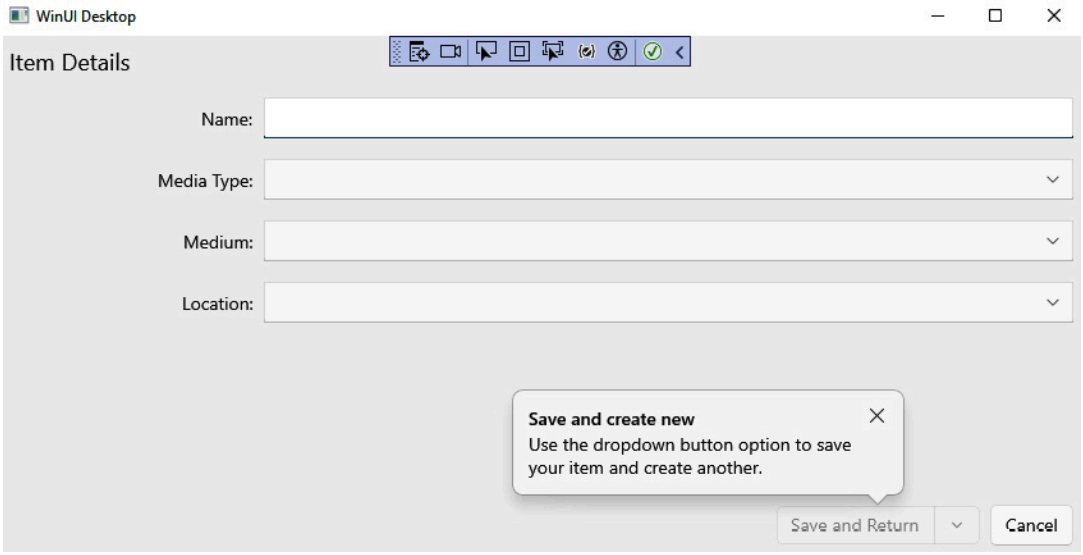

Figure 5.14 – Using the new TeachingTip control

Now, the application has a new feature and a great way to inform users of how to use it. Let's wrap up and discuss what we have learned in this chapter.

Summary

In this chapter, we explored many of the controls available in WinUI 3. We learned that the **WinUI 3 Gallery** application is a great tool for exploring the controls available to WinUI developers. We also explored some of the features in the Windows App SDK that are not part of WinUI. At the end, we learned about, and added, a couple of new WinUI controls to our application.

In the next chapter, we will learn more about services and will start persisting the data for our media items between sessions.

Questions

1. Which WinUI control can display Lottie animations?
2. Which WinUI control can display HTML content with the Chromium-based Microsoft Edge browser?
3. Can you create a WinUI 3 app with C++?

4. Which control would you use to educate users about a new feature?
5. Which application can you download from the Microsoft Store to learn about all the WinUI controls?
6. What type of control can be used to save and load user settings between sessions?
7. Which Windows App SDK feature would you use to notify users when your app has a new message to be viewed?